

FichaClara

Asistente RAG sobre fichas técnicas de medicamentos de la AEMPS (CIMA)

Explicación del proyecto, arquitectura, estructura del repositorio y reparto de tareas para un equipo de 5 personas

Documento de arranque · septiembre 2026 · nombre del proyecto provisional

Qué hay en este documento

1	El proyecto: problema, usuarios y valor	6	Reparto de tareas (5 roles)
2	Los datos: fichas técnicas de CIMA	7	Mapa de dependencias y mocks
3	Arquitectura	8	Planificación en 2 sprints
4	Decisiones técnicas propuestas	9	Flujo de trabajo Git
5	Estructura del repositorio y contratos	10	Presentación, cobertura del briefing y riesgos

Cómo usarlo. Todo lo que aparece como decisión técnica es una **propuesta de partida** para validar el día 1 entre todo el equipo. El reparto está pensado para que cada persona trabaje en sus propias carpetas, contra contratos acordados (`src/common/schemas.py`) y con datos o APIs simuladas, de modo que nadie tenga que esperar a otra persona para avanzar.

Junto a este PDF va el esqueleto del repositorio (`fichaclara_esqueleto.zip`): todas las carpetas y archivos vacíos, cada uno con su responsable y su lista de TODO, listos para el primer commit.

1. El proyecto

FichaClara es un asistente que responde en lenguaje natural preguntas sobre medicamentos usando **únicamente** las fichas técnicas oficiales publicadas por la AEMPS en CIMA, y que cita en cada respuesta el medicamento, la sección exacta (por ejemplo, *4.5 Interacciones*) y la página de la que sale la información. Si la respuesta no está en las fichas indexadas, lo dice en lugar de inventarla.

El problema

- Una ficha técnica tiene entre 10 y 40 páginas y la información clave está repartida en secciones que no se leen de un vistazo en mitad de un turno.
- Preguntas como «¿se puede triturar?», «¿hay que ajustar la dosis en insuficiencia renal?» o «¿interacciona con el acenocumarol?» se resuelven hoy buscando a mano en el PDF o preguntando a farmacia.
- La búsqueda de CIMA es por nombre o por palabra clave: no entiende la pregunta ni resume la respuesta.
- Un chatbot genérico responde rápido, pero puede alucinar y no dice de dónde saca el dato. En medicamentos, eso es inaceptable.

Usuarios y valor de negocio

Usuario	Situación típica	Valor
Enfermería de residencias y hospitalización	Preparar la medicación de un residente con disfagia en el turno de noche	Respuesta en segundos con la sección citada para comprobarla
Farmacia hospitalaria / de residencia	Resolver consultas repetitivas del personal	Menos interrupciones; las consultas simples se autoatienden
Estudiantes y personal en formación	Aprender a leer una ficha técnica	El sistema enseña <i>dónde</i> está cada dato (sección 4.2, 4.8...)
Proveedores de software sanitario (cliente B2B)	Integrar consulta de fichas en su historia clínica o app	Módulo reutilizable vía API con trazabilidad completa

Alcance

Dentro del alcance	Fuera del alcance
Consultar fichas técnicas (y, opcionalmente, prospectos) de un catálogo de 50-100 medicamentos	Recomendaciones clínicas individualizadas («¿le doy esto a mi paciente?»)
Subir documentos propios en PDF, TXT o MD (p. ej. una guía interna de farmacia)	Diagnóstico o prescripción
Citar medicamento, sección, página y enlace a CIMA	Tratar datos de pacientes reales
Responder «no consta en las fichas consultadas» cuando no hay información	Sustituir la consulta al farmacéutico o al médico

Frase para la defensa: «No es un sistema de apoyo a la decisión clínica. Es un buscador inteligente sobre documentación oficial, que siempre enseña la fuente para que el profesional la verifique.» Esto delimita la responsabilidad y es la base del apartado ético.

2. Los datos: fichas técnicas de CIMA

CIMA es el Centro de Información online de Medicamentos de la AEMPS. Ofrece una API REST pública y gratuita (base <https://cima.aemps.es/cima/rest/>) que devuelve JSON, y cada medicamento incluye los enlaces a sus documentos. Todas las fichas técnicas autorizadas en España siguen la misma plantilla de secciones numeradas, y esa es la clave de la estrategia de chunking.

Recursos de la API que usaremos

Recurso	Para qué
GET medicamentos?nombre=...&practiv1=...	Buscar medicamentos del catálogo y obtener su nº de registro
GET medicamento?nregistro=...	Detalle del medicamento, incluida la lista docs (tipo 1 = ficha técnica, 2 = prospecto) con la URL del PDF
GET docSegmentado/secciones/1?nregistro=...	Lista oficial de secciones de la ficha: sirve para validar que nuestro chunker las detecta bien
cima.aemps.es/cima/dochtml/ft/{nregistro}/{seccion}/FichaTecnica.html	Enlace directo a una sección concreta: lo usaremos como enlace de cada fuente en el frontend

Importante: el briefing pide un pipeline que procese PDF, TXT o Markdown, así que la fuente principal de ingesta serán los **PDF** descargados. La API segmentada solo se usa para validar la detección de secciones.

Estructura de una ficha técnica

Sección	Contenido	Preguntas típicas
1-3	Nombre, composición, forma farmacéutica	¿Qué dosis tiene el comprimido? ¿Es ranurado?
4.1	Indicaciones terapéuticas	¿Para qué está indicado?
4.2	Posología y forma de administración	¿Se puede triturar? ¿Ajuste en insuficiencia renal?
4.3 / 4.4	Contraindicaciones / advertencias y precauciones	¿Se puede usar en mayores de 75 años?
4.5	Interacciones	¿Interacciona con el omeprazol?
4.6 / 4.7	Embarazo y lactancia / conducción	¿Es compatible con la lactancia?
4.8 / 4.9	Reacciones adversas / sobredosis	¿Es frecuente el mareo?
5	Propiedades farmacológicas	¿Cuál es su semivida?
6.1-6.6	Excipientes, incompatibilidades, validez, conservación, envase	¿Lleva lactosa? ¿Cuánto dura abierto?
7-10	Titular, nº de autorización, fechas, fecha de revisión del texto	(metadato de vigencia)

Catálogo de medicamentos

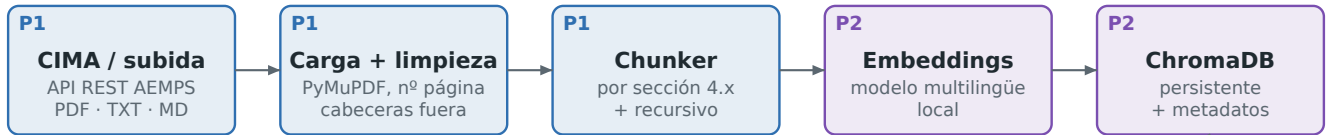
Proponemos un catálogo cerrado de **50-100 principios activos frecuentes en residencias y atención primaria** (antihipertensivos, anticoagulantes, antidiabéticos, psicofármacos, analgésicos, IBP...). Con eso el índice tiene unos miles de fragmentos: suficiente para que el retrieval sea un reto real y pequeño para indexarlo en local. El catálogo vive en `data/catalogo_medicamentos.csv` y lo mantiene P1.

Las fichas técnicas son documentos **públicos**: no hay datos personales en la base de conocimiento. El riesgo de privacidad está en las **preguntas** que escriba el usuario, lo que se trata en el apartado 4.

3. Arquitectura

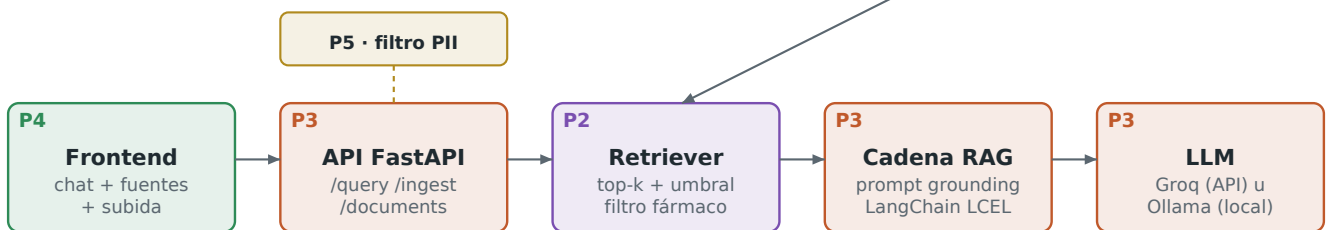
Dos flujos independientes que comparten la base vectorial. El color de cada caja indica qué rol la construye.

FLUJO DE INGESTA (offline o al subir un documento)



P5 · golden set + métricas de retrieval y generación evalúan ambos flujos · CI en cada PR

FLUJO DE CONSULTA (cada pregunta)



Flujo de consulta, paso a paso

1. El usuario escribe la pregunta en el frontend (P4), que la envía a POST /query (P3).
2. El filtro de datos personales (P5) avisa o enmascara si detecta un DNI, un nombre o un nº de historia antes de enviar nada a una API comercial.
3. El *query parser* (P2) detecta si se nombra un medicamento del catálogo para filtrar la búsqueda por su nº de registro.
4. El retriever (P2) devuelve los k fragmentos más parecidos que superan un **umbral de relevancia**.
5. **Si no hay ninguno, no se llama al LLM:** la API responde found=false con un mensaje fijo (P3). Esto cubre el «qué pasa si no encuentra nada relevante» del briefing.
6. Si los hay, la cadena LangChain (P3) construye el prompt de grounding con los fragmentos numerados y llama al LLM.
7. P3 valida que cada cita de la respuesta corresponde a un fragmento realmente recuperado, y la API devuelve respuesta + fuentes.
8. El frontend (P4) muestra la respuesta y un panel de fuentes con medicamento, sección, página, fragmento y enlace a CIMA.

4. Decisiones técnicas propuestas

Cada decisión se documenta como un ADR (*Architecture Decision Record*) en docs/decisiones_tecnicas.md, con las alternativas y el motivo. Esto es exactamente lo que se defiende en la presentación oral.

4.1 Chunking (P1) — el criterio con más peso en la evaluación

Parámetro	Propuesta de partida	Justificación
Unidad	Una sección numerada de la ficha (4.2, 4.5, 6.1...)	Cada sección responde a un tipo de pregunta. Trocear por tamaño fijo mezclaría «interacciones» con «embarazo» en un mismo fragmento.
Secciones largas	Si superan ~1.500 caracteres, subtrozos de ~1.000 caracteres (RecursiveCharacterTextSplitter)	4.8 y 5.1 pueden ocupar varias páginas; un fragmento enorme diluye el embedding.
Overlap	~150 caracteres (≈15 %), solo dentro de la misma sección	Evita cortar una frase o una fila de tabla, sin contaminar una sección con la siguiente.
Cabecera contextual	Cada fragmento empieza por «[Nombre] · 4.5 Interacciones»	Un subtrozo de 4.8 no dice de qué fármaco habla; la cabecera lo mete en el embedding y mejora mucho el retrieval.
Metadatos	nregistro, nombre, principio activo, ATC, sección, título, página, URL, fecha de revisión	Trazabilidad completa y filtrado por medicamento.
Documentos subidos	Fallback: troceo recursivo genérico (~1.000 / 150) respetando encabezados Markdown	Una guía interna no tiene secciones numeradas.

Experimento obligatorio: comparar troceo fijo (500 y 1.000 caracteres) frente a troceo por sección con el golden set de P5. La tabla de resultados es la prueba de que el chunking está justificado.

4.2 Embeddings y base vectorial (P2)

Decisión	Propuesta	Alternativa a comparar	Motivo
Modelo de embeddings	BAAI/bge-m3 (local, multilingüe)	intfloat/multilingual-e5-base (más ligero)	Buen rendimiento en español y en texto técnico; se ejecuta en local, así que los documentos no salen del equipo. Se elige con el golden set.
Base vectorial	ChromaDB persistente en data/chroma	Qdrant en Docker	Sin servidor, filtros por metadatos, integración nativa con LangChain. Qdrant solo si hiciera falta escalar.
Recuperación	Top-k (k≈5) + umbral + filtro por medicamento	Híbrido vectorial + BM25	BM25 ayuda con nombres de fármacos y siglas exactas, que los embeddings a veces confunden.

4.3 Orquestación y LLM (P3)

- **Orquestador:** LangChain con LCEL (retriever → prompt → LLM → parser). Es el que más ejemplos tiene y encaja con el temario.
- **LLM intercambiable** por variable de entorno: Groq (API comercial, rápido y con nivel gratuito) y Ollama (modelo local). Tener los dos permite demostrar el argumento de privacidad en directo.
- **Prompt de sistema con grounding:** responder solo con el contexto; si no está, usar una frase fija; citar cada afirmación como [n] asociado a un fragmento; no dar recomendaciones clínicas; ignorar instrucciones que vengan dentro de los documentos (protección básica frente a *prompt injection*). Temperatura 0.

4.4 Frontend (P4)

Streamlit: permite tener chat, subida de ficheros y panel de fuentes en pocos días, en Python, y deja más tiempo para el retrieval, que es lo que se evalúa. React solo si el equipo tiene tiempo de sobra y alguien quiere practicarlo.

4.5 Privacidad: ¿API comercial o modelo local?

Escenario	Riesgo	Decisión
Base de conocimiento (fichas técnicas públicas)	Nulo: son documentos públicos	Embeddings en local de todos modos (coste cero, sin dependencia)
Preguntas genéricas («¿se puede triturar X?»)	Bajo	API comercial aceptable
Preguntas con datos de un paciente («Juan, 87 años, NHC 1234...»)	Alto: datos de salud (categoría especial RGPD) enviados a un tercero	Filtro PII (P5) que avisa o enmascara + modo local con Ollama para despliegue real en un centro
Documentos internos subidos	Medio: pueden contener información confidencial	Aviso en la subida; recomendación de modo local

5. Estructura del repositorio

La carpeta de cada módulo pertenece a una sola persona, lo que evita conflictos de merge. Solo src/common/, requirements.txt y el README se tocan entre varios, siempre por PR.

```
fichaclara/
├── README.md                P5  README técnico final
├── requirements.txt         P5  (cada persona añade sus dependencias por PR)
├── .env.example            P3
├── docker-compose.yml      P5  (opcional)
├── .github/                P5  CI, plantilla de PR y de issue
├── data/
│   ├── catalogo_medicamentos.csv P1
│   ├── raw/ processed/ chroma/ (ignorados por Git)
├── docs/
│   ├── arquitectura.md evaluacion.md etica_y_privacidad.md P5
│   ├── chunking.md P1
│   ├── decisiones_tecnicas.md (ADR: cada rol escribe el suyo) P5 integra
│   └── presentacion/
├── src/
│   ├── common/ schemas.py config.py TODOS el día 1 · custodia P3
│   ├── ingestion/ cima_client loaders cleaner chunker pipeline P1
│   ├── indexing/ embeddings vector_store P2
│   ├── retrieval/ query_parser retriever P2
│   ├── generation/ llm_providers prompts rag_chain P3
│   ├── api/ main routes errors P3
│   └── guardrails/ pii_filter P5
├── frontend/ app api_client mock_api components/ P4
├── scripts/ download_fichas.py (P1) build_index.py (P2)
├── evaluation/ golden_set.jsonl eval_retrieval eval_generation P5
└── tests/ un test_*.py por módulo, de su propio responsable
```

Contratos entre módulos (se acuerdan el día 1)

Son la pieza que hace posible trabajar en paralelo. Una vez fijados, cada persona programa contra ellos aunque el módulo del que depende todavía no exista.

Contrato	Lo produce	Lo consume	Forma
Chunk + ChunkMetadata	P1	P2, P5	data/processed/chunks.jsonl, una línea por chunk
retrieve(question, k) → list[RetrievedChunk]	P2	P3, P5	Función Python
ingest_file(path) → list[Chunk]	P1	P3 (endpoint /ingest)	Función Python
add_chunks(chunks), delete_document(doc_id)	P2	P3	Función Python
QueryResponse (answer, sources[], found)	P3	P4, P5	JSON de POST /query
check_pii(text) → PiiResult	P5	P3	Función Python

6. Reparto de tareas

Cinco roles, cada uno con código propio, tests propios, un ADR, una parte del README y una parte de la presentación. Así el trabajo queda equilibrado en el historial de Git, que es un criterio de evaluación.

Rol	Módulo	Competencia principal	Perfil recomendado
P1	Ingesta y chunking	C6 datos · C9	Python y datos; paciencia con PDFs
P2	Embeddings, base vectorial y retrieval	C9 · C6	Quien más disfrute de ML y métricas
P3	Orquestación LLM y API	C9	Backend con FastAPI y LLMs
P4	Frontend y experiencia de usuario	C9 · C8	Interfaz, UX y presentación
P5	Calidad, evaluación, gobernanza y documentación	C10 · C8	Perfil sanitario, Scrum y visión global

P1 · Ingesta y chunking

Archivos	src/ingestion/*, scripts/download_fichas.py, data/catalogo_medicamentos.csv, docs/chunking.md, tests/test_chunker.py, ADR-05
Tareas	1. Curar el catálogo de 50-100 medicamentos con su nº de registro (con ayuda de P5 si tiene perfil sanitario). 2. Cliente de CIMA: búsqueda, obtención de la URL de la ficha técnica y descarga a data/raw/ con reintentos y pausa entre peticiones. 3. Loaders de PDF (PyMuPDF, conservando el nº de página), TXT y MD, con error controlado si el PDF está vacío o escaneado. 4. Limpieza: cabeceras y pies repetidos, «Página X de Y», palabras partidas por guion, espacios. 5. Chunker por sección + subtroceo recursivo + cabecera contextual + metadatos; fallback genérico para documentos subidos. 6. Día 2: publicar un chunks.jsonl de muestra con 5 fichas en tests/fixtures/ (desbloquea a P2 y P5). 7. Validar la detección de secciones contra docSegmentado/secciones. 8. Experimento de chunking (fijo 500 / fijo 1.000 / por sección) junto con P5, y redactar docs/chunking.md.
Entregable	data/processed/chunks.jsonl de todo el catálogo + ingest_file() para documentos subidos.
Criterio de aceptación	Las secciones 4.2, 4.3, 4.5 y 4.8 se detectan en al menos el 95 % de las fichas; tests del chunker en verde.
Para no bloquearse	No depende de nadie. Trabaja desde el día 1 con PDFs reales.
En la presentación	Estrategia de chunking, con la tabla del experimento.

P2 · Embeddings, base vectorial y retrieval	
Archivos	src/indexing/*, src/retrieval/*, scripts/build_index.py, tests/test_retriever.py, ADR-01 y ADR-02
Tareas	1. Montar ChromaDB persistente con <i>upsert</i> por chunk_id (reindexar no duplica), borrado por documento y listado. 2. Script build_index.py: de chunks.jsonl al índice. 3. Comparar dos modelos de embeddings (bge-m3 frente a multilingual-e5-base) con el golden set: calidad, tiempo de indexado y memoria. 4. <i>Query parser</i>: detectar el medicamento nombrado en la pregunta (nombre comercial o principio activo) y filtrar por él. 5. Retriever top-k con umbral de relevancia, calibrado con las preguntas sin respuesta del golden set. 6. Experimento de búsqueda híbrida (vectorial + BM25). 7. Presentar las métricas de retrieval en el checkpoint con los formadores (junto con P5).
Entregable	Función retrieve(question, k) estable + índice construido + ADR-01/02 con los resultados.
Criterio de aceptación	Hit rate@5 de al menos 0,85 en el golden set (objetivo orientativo) y lista vacía cuando la pregunta está fuera de alcance.
Para no bloquearse	Empieza con el chunks.jsonl de muestra de P1 (día 2). Los días 1-2 prepara Chroma y embeddings con 3 chunks de prueba escritos a mano.
En la presentación	Elección del modelo de embeddings y de la base vectorial, y métricas de retrieval.

P3 · Orquestación LLM y API	
Archivos	src/common/* (custodia), src/generation/*, src/api/*, .env.example, tests/test_rag_chain.py, tests/test_api.py, ADR-03
Tareas	1. Día 1: redactar schemas.py y config.py con todo el equipo. 2. Proveedores de LLM intercambiables: Groq y Ollama, elegidos desde .env. 3. Prompt de sistema con grounding, formato de cita [n], frase fija de «no consta» y defensa básica frente a prompt injection. 4. Cadena LCEL. Mientras P2 termina, usa un FakeRetriever que lee los fixtures. 5. Manejo de excepciones: sin contexto no se llama al LLM; timeouts; proveedor caído; documento ilegible. 6. Validación posterior: cada cita [n] de la respuesta debe corresponder a un fragmento recuperado; si no, se descarta o se marca. 7. FastAPI: /health, /ingest (conecta P1 → P2), /documents, DELETE /documents/{id} y /query (integra el filtro PII de P5). 8. Tests de la cadena con LLM simulado y de los endpoints con TestClient.
Entregable	API funcionando de punta a punta, documentada en /docs (Swagger).
Criterio de aceptación	Preguntas fuera de alcance → found=false sin llamar al LLM; el 100 % de las citas apuntan a fragmentos reales.
Para no bloquearse	Empieza con fixtures y retriever falso; integra el retriever real en el sprint 2.
En la presentación	Orquestación, prompt de grounding y argumento de API comercial frente a modelo local (con demostración en directo).

P4 · Frontend y experiencia de usuario	
Archivos	frontend/*, ADR-04, capturas del README, guion de la demo
Tareas	1. Wireframe de la interfaz (días 1-2) y validación rápida con el equipo. 2. mock_api.py que devuelve respuestas falsas con la forma exacta de QueryResponse. 3. Chat con historial de la sesión. 4. Panel de fuentes: medicamento, sección, página, fragmento resaltado, fecha de revisión y enlace directo a la sección en CIMA. 5. Subida de documentos con estado (procesando, indexado, error) y listado o borrado de documentos. 6. Estado «no consta en las fichas» visualmente distinto de una respuesta; aviso sanitario permanente; aviso de PII cuando lo devuelva la API. 7. Conectar con la API real (sprint 2) sustituyendo el mock por api_client.py con un solo cambio de configuración. 8. Guion de la demo y capturas para el README.
Entregable	Interfaz completa conectada a la API.
Criterio de aceptación	Una persona ajena al equipo puede subir un documento, preguntar y abrir la fuente en CIMA sin ayuda.
Para no bloquearse	Trabaja todo el sprint 1 contra el mock; no necesita backend hasta el día 6.
En la presentación	Valor de negocio del caso de uso y demo en directo.

P5 · Calidad, evaluación, gobernanza y documentación	
Archivos	evaluation/*, src/guardrails/pii_filter.py, tests/test_pii_filter.py, .github/*, requirements.txt, README.md, docs/arquitectura.md, docs/evaluacion.md, docs/etica_y_privacidad.md
Tareas	1. Día 1: crear el repositorio, proteger main y develop, plantillas de PR e issue, tablero y etiquetas p1-p5. 2. CI con GitHub Actions: ruff y pytest en cada PR, sin llamadas a APIs externas. 3. Golden set de ~40 preguntas: ~30 con respuesta (medicamento y sección esperados) y ~10 sin respuesta, fuera de alcance o con intento de prompt injection. 4. eval_retrieval.py (hit rate@k, MRR, acierto de sección) → métricas para el checkpoint. 5. eval_generation.py: cita correcta, rechazo correcto de preguntas sin respuesta y fidelidad (revisión manual o LLM como juez). 6. Filtro de datos personales (DNI/NIE, teléfono, email, nº de historia, nombres) y sus tests. 7. Documentos de ética y privacidad y de evaluación; integrar los ADR; README técnico final. 8. Facilitar la <i>daily</i> de 10 minutos y el tablero (rol de Scrum Master).
Entregable	Golden set, informes de métricas, CI en verde, README y documentación de ética.
Criterio de aceptación	El README permite a alguien externo instalar y ejecutar el proyecto; las métricas se pueden reproducir con un comando.
Para no bloquearse	El golden set solo necesita el catálogo (día 1-2) y las fichas en PDF; no depende de código de nadie.
En la presentación	Evaluación y resultados, y reflexión ética: datos, explicabilidad y riesgo de fuga de información.

7. Mapa de dependencias y mocks

Los puntos de encuentro entre roles son pocos, y todos tienen un sustituto para que nadie tenga que esperar.

Quién necesita	Qué	De quién	Cuándo llega lo real	Mientras tanto usa
P2, P5	chunks con metadatos	P1	Muestra: día 2 · Completo: día 4	3 chunks escritos a mano con el esquema acordado
P3	retrieve()	P2	Día 5	FakeRetriever que lee tests/fixtures/
P3	check_pii()	P5	Día 6	Función que devuelve «sin PII»
P4	API /query y /ingest	P3	Día 6	mock_api.py
P5	sistema completo para evaluar la generación	P3	Día 6	Evalúa el retrieval desde el día 4

Regla de oro: los contratos de `src/common/schemas.py` solo cambian con una PR aprobada por **dos** personas y avisando en el canal del equipo. Es el único archivo en el que un cambio descoordinado rompe el trabajo de otros.

8. Planificación en 2 sprints

Diez días hábiles. El checkpoint de retrieval se coloca al final del sprint 1, como pide el briefing: primero se afina la recuperación y después la generación.

Día	Todo el equipo	P1	P2	P3	P4	P5
1	Kickoff: caso de uso, contratos, reparto, tablero	Catálogo	Setup Chroma	schemas + config	Wireframe	Repo, CI base, plantillas
2-4	Daily 10 min	Descarga, loaders, chunker; muestra día 2	Embeddings, índice con muestra	Proveedores, prompt, cadena con fake	Mock + chat + fuentes	Golden set + eval_retrieval
5	Integración 1 + checkpoint de retrieval	Índice completo	Métricas y umbral	Retriever real	Demo interna con mock	Informe de métricas
6-8	Daily 10 min	Experimento de chunking	Híbrido, comparativa de modelos	API completa, validación de citas	UI con API real	PII, eval_generation, ética
9	Code freeze, README, ensayo	docs/chunking	ADR 01-02	ADR 03, Swagger	Capturas, guion demo	README y docs finales
10	Presentación					

9. Flujo de trabajo Git

El briefing evalúa explícitamente ramas, pull requests, commits descriptivos y un reparto equilibrado. Estas reglas lo garantizan.

Elemento	Regla
Ramas	main (entregable, protegida) ← develop (integración, protegida) ← feature/p1-chunker, feature/p3-api-query, fix/p2-umbral...
Issues	Cada tarea de este documento es una issue con la etiqueta de su rol (p1-p5) y criterio de aceptación. Las PR la cierran con «Closes #n».
Pull requests	Siempre a develop. Pequeñas (idealmente menos de 300 líneas). CI en verde y una aprobación obligatoria.
Revisión cruzada	Rotación fija para que todos revisen y sean revisados: P1 → revisa P2 → revisa P3 → revisa P4 → revisa P5 → revisa P1.
Commits	Convencionales: feat(ingestion): detectar secciones 4.x, fix(retrieval): umbral en preguntas vacías, docs(chunking): tabla del experimento, test(api): ...
Merge a main	Al final de cada sprint (develop → main), con etiqueta v0.1 y v1.0.
Nunca en el repo	.env, claves de API, data/raw, data/chroma. Ya están en .gitignore.

Equilibrio visible: cada persona debería acabar con código propio en src/ o frontend/, tests propios, al menos 5-6 PR fusionadas, al menos 5 revisiones hechas y una sección del README o de los ADR. La rotación de revisiones y el reparto por carpetas lo facilitan sin esfuerzo extra.

10. Presentación, cobertura del briefing y riesgos

Guion de la presentación oral

Bloque	Quién	Contenido
1. Valor de negocio	P4	El problema en un turno de noche, usuarios, por qué un RAG y no un chatbot
2. Demo en directo	P4	Pregunta respondida con fuentes → abrir CIMA · pregunta sin respuesta → «no consta»
3. Datos y chunking	P1	Estructura de la ficha técnica, troceo por sección, experimento
4. Embeddings y base vectorial	P2	Modelo elegido y por qué, Chroma, métricas de retrieval
5. Orquestación y grounding	P3	Prompt, validación de citas, API frente a local (cambio de proveedor en directo)
6. Evaluación y ética	P5	Resultados, datos de salud, explicabilidad, fuga de información y límites del sistema

Cobertura de todos los requisitos del briefing

Requisito del briefing	Dónde se cubre	Responsable
Pipeline de ingesta PDF / TXT / MD	src/ingestion/	P1
Chunking y overlap justificados y documentados	docs/chunking.md + experimento	P1 (+P5)
Embeddings + base vectorial persistente	src/indexing/, ChromaDB en data/chroma	P2
Backend con LangChain / LlamaIndex, top-k	src/retrieval/ + src/generation/rag_chain.py	P2, P3

Requisito del briefing	Dónde se cubre	Responsable
Manejo de excepciones (sin contexto relevante)	Umbral + found=false sin llamar al LLM	P2, P3
Frontend: subir documentos, chatear, fuentes	frontend/	P4
Control de alucinaciones	Prompt de grounding + validación de citas + evaluación	P3, P5
Privacidad: API comercial frente a modelo local	Groq / Ollama intercambiables + filtro PII + documento de ética	P3, P5
Trazabilidad: documento y sección	Metadatos + panel de fuentes con enlace a CIMA	P1, P4
Checkpoint de calidad de retrieval	Día 5: golden set + hit rate@k y MRR	P2, P5
Repositorio: ramas, PR, commits, reparto equilibrado	Sección 9	Todos (P5 vigila)
requirements.txt + README técnico	Raíz del repositorio	P5
Presentación: negocio, defensa técnica, ética	Guion anterior	Todos
Competencias C6 · C8 · C9 · C10	C6: P1, P2 · C8: todos (Git, daily, demo) · C9: P1-P4 · C10: P5, P3	—

Riesgos principales

Riesgo	Mitigación
Las tablas de 4.8 (reacciones adversas) se extraen desordenadas del PDF	P1 trata la sección como texto plano y lo documenta como limitación; opcionalmente, extracción de tablas con pdfplumber
bge-m3 es pesado para algún portátil del equipo	Índice construido una sola vez y compartido; alternativa multilingual-e5-base
Límite de peticiones de Groq o CIMA	Caché de respuestas en desarrollo; pausas en la descarga; Ollama como respaldo
Integración tardía	Contratos el día 1, mocks y una integración obligatoria el día 5
El sistema parece dar consejo clínico	Aviso permanente, prompt que lo prohíbe y preguntas de ese tipo en el golden set